

Целочисленная арифметика многократной точности

Филибер Никола

7 Март 2026, Москва, Россия

Российский Университет Дружбы Народов

Цель работы

Реализация алгоритмов многократной точности для целочисленной арифметики.

Задание

Задание

1. Реализовать алгоритм сложения неотрицательных целых чисел.
2. Реализовать алгоритм вычитания неотрицательных целых чисел.
3. Реализовать алгоритм умножения неотрицательных целых чисел столбиком.
4. Реализовать алгоритм умножения методом быстрого столбика.
5. Реализовать алгоритм деления неотрицательных целых чисел.

Теоретическое введение

Рассмотрим алгоритмы для выполнения арифметических операций с большими целыми числами. Будем считать, что число записано в b -ичной системе счисления, b - натуральное число, $b \geq 2$. Натуральное n -разрядное число будем записывать в виде: $u = u_1 u_2 \dots u_n$

Алгоритм сложения неотрицательных целых чисел

*Вход. Два неотрицательных числа $u = u_1u_2 \dots u_n$ и $v = v_1v_2 \dots v_n$; разрядность чисел n ; основание системы счисления b .

*Выход. Сумма $w = w_0w_1 \dots w_n$, где w_0 - цифра переноса, всегда равная 0 либо 1.

1. Присвоить $j = n, k = 0$ (j идет по разрядам, k следит за переносом).
2. Присвоить $w_j = (u_j + v_j + k) \pmod{b}$, где $k = \left\lfloor \frac{u_j + v_j + k}{b} \right\rfloor$.
3. Присвоить $j = j - 1$. Если $j > 0$, то возвращаемся на шаг 2; если $j = 0$, то присвоить $w_0 = k$ и результат: w .

Алгоритм вычитания неотрицательных целых чисел

*Вход. Два неотрицательных числа $u = u_1u_2 \dots u_n$ и $v = v_1v_2 \dots v_n$, $u > v$; разрядность чисел n ; основание системы счисления b .

*Выход. Разность $w = w_0w_1 \dots w_n = u - v$.

1. Присвоить $j = n$, $k = 0$ (k – заём из старшего разряда).
2. Присвоить $w_j = (u_j - v_j + k) \pmod{b}$;
 $k = \left\lfloor \frac{u_j - v_j + k}{b} \right\rfloor$.
3. Присвоить $j = j - 1$. Если $j > 0$, то возвращаемся на шаг 2; если $j = 0$, то результат: w .

Алгоритм умножения неотрицательных целых чисел столбиком

*Вход. Числа $u = u_1 u_2 \dots u_n$, $v = v_1 v_2 \dots v_m$; основание системы счисления b .

*Выход. Произведение $w = uv = w_1 w_2 \dots w_{m+n}$.

1. Выполнить присвоения:

$w_{m+1} = 0, w_{m+2} = 0, \dots, w_{m+n} = 0, j = m$ (j перемещается по номерам разрядов числа v от младших к старшим).

2. Если $v_j = 0$, то присвоить $w_j = 0$ и перейти на шаг 6.

3. Присвоить $i = n, k = 0$ (значение i идет по номерам разрядов числа u , k отвечает за перенос).

4. Присвоить $t = u_i \cdot v_j + w_{i+j} + k, w_{i+j} = t \pmod{b}, k = \lfloor \frac{t}{b} \rfloor$.
5. Присвоить $i = i - 1$. Если $i > 0$, то возвращаемся на шаг 4, иначе присвоить $w_j = k$.
6. Присвоить $j = j - 1$. Если $j > 0$, то вернуться на шаг 2. Если $j = 0$, то результат: w .

Алгоритм умножения методом быстрого столбика

*Вход. Числа $u = u_1 u_2 \dots u_n$, $v = v_1 v_2 \dots v_m$; основание системы счисления b .

*Выход. Произведение $w = uv = w_1 w_2 \dots w_{m+n}$.

1. Присвоить $t = 0$.
2. Для s от 0 до $m + n - 1$ с шагом 1 выполнить шаги 3 и 4.
3. Для i от 0 до s с шагом 1 выполнить присвоение $t = t + u_{n-i} \cdot v_{m-s+i}$.
4. Присвоить $w_{m+n-s} = t \pmod{b}$, $t = \left\lfloor \frac{t}{b} \right\rfloor$. Результат: w .

Алгоритм деления неотрицательных целых чисел

*Вход. Числа $u = u_n \dots u_1 u_0$,
 $v = v_t \dots v_1 v_0, n \geq t \geq 1, v_t \neq 0$.

*Выход. Частное $q = q_{n-t} \dots q_0$, остаток $r = r_t \dots r_0$.

1. Для j от 0 до $n - t$ присвоить $q_j = 0$.
2. Пока $u \geq vb^{n-t}$, выполнять:
 $q_{n-t} = q_{n-t} + 1, u = u - vb^{n-t}$.
3. Для $i = n, n - 1, \dots, t + 1$ выполнять пункты 3.1 – 3.4:
3.1. если $u_i \geq v_t$, то присвоить $q_{i-t-1} = b - 1$, иначе
присвоить $q_{i-t-1} = \frac{u_i b + u_{i-1}}{v_t}$. 3.2. пока
 $q_{i-t-1}(v_t b + v_{t-1}) > u_i b^2 + u_{i-1} b + u_{i-2}$ выполнять
 $q_{i-t-1} = q_{i-t-1} - 1$. 3.3. присвоить
 $u = u - q_{i-t-1} b^{i-t-1} v$. 3.4. если $u < 0$, то присвоить
 $u = u + vb^{i-t-1}, q_{i-t-1} = q_{i-t-1} - 1$.
4. $r = u$. Результат: q и r .

Выполнение лабораторной работы

Реализация Алгоритма сложения неотрицательных целых чисел

```
[9]: def trim_leading_zeros(arr: list[int]) -> list[int]:  
    """  
    Удаляет ведущие нули, но оставляет хотя бы один ноль.  
    """  
    i = 0  
    while i < len(arr) - 1 and arr[i] == 0:  
        i += 1  
    return arr[i:]
```

Рис. 1: trim_leading_zeros function

```
[10]: def pad_left(arr: list[int], n: int) -> list[int]:  
    """  
    Дополняет число слева нулями до длины n.  
    """  
    if len(arr) == n:  
        return arr  
    i = len(arr) - 1  
    new_arr = [0] * n  
    j = n - 1  
    while i >= 0:  
        new_arr[j] = arr[i]  
        i -= 1  
        j -= 1  
  
    while j >= 0:  
        new_arr[j] = 0  
        j -= 1  
  
    return new_arr
```

Рис. 2: pad_left function

```
def add(u:list[int], v:list[int], b:int)-> list[int]:
    """
    Вход:
        u = u1u2...un
        v = v1v2...vn
        b - основание системы счисления

    Выход:
        w = w0w1...wn
    """

    n = max(len(u), len(v))
    u = pad_left(u, n)
    v = pad_left(v, n)
    w = [0] * (n + 1)

    # Шаг 1: j := n, k := 0
    j = n
    k = 0

    # Шаги 2-3
    while True:
        u_j = u[j-1]
        v_j = v[j-1]

        w[j] = (u_j + v_j + k) % b
        k = (u_j + v_j + k) // b

        # Шаг 3:
        j = j - 1
        if j > 0:
            continue

        if j == 0:
            w[0] = k
            break

    return trim_leading_zeros(w)
```



Рис. 3: add function

```
u = [1,2,3]
v = [4,5,6]
result = add(u, v, 10) |
print("result = ", result)

result = [5, 7, 9]
```



Рис. 4: result

Реализация Алгоритма вычитания неотрицательных целых чисел

```
def subtract(u:list[int], v:list[int], b:int)-> list[int]:
```

```
    """
```

```
    Вход:
```

```
        u = u1u2...un
```

```
        v = v1v2...vn
```

```
        b – основание системы счисления
```

```
        u > v
```

```
    Выход:
```

```
        w = w0w1...wn = u - v
```

```
    """
```

```
n = max(len(u), len(v))
```

```
u = pad_left(u, n)
```

```
v = pad_left(v, n)
```

```
w = [0] * (n + 1)
```

```
j = n
```

```
k = 0
```

```
while True:
```

```
    u_j = u[j-1]
```

```
    v_j = v[j - 1]
```

```
    w[j] = (u_j - v_j + k) % b
```

```
    k = (u_j - v_j + k) // b
```

```
    j = j - 1
```

```
    if j > 0:
```

```
        continue
```

```
    if j == 0:
```

```
        return trim_leading_zeros(w)
```

Рис. 5: subtract function

```
u = [4,5,6]
v = [1,2,3]
result = subtract(u, v, 10)
print("result = ", result)

result = [3, 3, 3]
```



Рис. 6: result

Реализация Алгоритма умножения неотрицательных целых чисел столбиком

```
def multiply(u:list[int], v:list[int], b:int) -> list[int]:
    """
    Вход:
        u = u1u2...un
        v = v1v2...vn
        b - основание системы счисления

    Выход:
        w = uv = w1w1...wm+n
    """
    u = trim_leading_zeros(u)
    v = trim_leading_zeros(v)

    n = len(u)
    m = len(v)

    # Шаг 1:
    w = [0] * (m + n)
    j = m

    # u_i соответствует u[i-1]
    # v_j соответствует v[j-1]
    # w_k соответствует w[k-1]
    while True:
        # Шаг 2:
        if v[j - 1] == 0:
            w[m - j] = 0
        else:
            # Шаг 3:
            i = n
            k = 0
            while True:
                # Шаги 4-5
                u_i = u[i - 1]
                v_j = v[j - 1]
                w_index = i + j - 1
                t = u_i * v_j + w[w_index] + k
                w[w_index] = t % b
                k = t // b
```

```
        i = i - 1
        if i > 0:
            continue
        else:
            w[j-1] = k
            break

# War 6
j = j - 1
if j == 0:
    return trim_leading_zeros(w)
```

Рис. 8: multiply

```
u = [1,2,3]
v = [4,5]
b = 10
result = multiply(u, v, b)
print("result = ", result)
```

result = [5, 5, 3, 5]

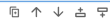


Рис. 9: result

Реализация Алгоритма умножения методом быстрого столбика

```
def multiply_fast(u:list[int], v:list[int], b:int) -> list[int]:
```

```
    """
```

```
    Вход:
```

```
        u = u1u2...un
```

```
        v = v1v2...vn
```

```
        b - основание системы счисления
```

```
    Выход:
```

```
        w = uv = w1w2...wm+n
```

```
    """
```

```
    u = trim_leading_zeros(u)
```

```
    v = trim_leading_zeros(v)
```

```
    n = len(u)
```

```
    m = len(v)
```

```
    w = [0] * (m + n)
```

```
    t = 0
```

```
    for s in range(0, m + n):
```

```
        for i in range(0, s + 1):
```

```
            u_pos = n - i
```

```
            v_pos = m - s + i
```

```
            if 1 <= u_pos <= n and 1 <= v_pos <= m:
```

```
                t = t + u[u_pos - 1] * v[v_pos - 1]
```

```
        w_index = m + n - s - 1
```

```
        w[w_index] = t % b
```

```
        t = t // b
```

```
    return trim_leading_zeros(w)
```

Рис. 10: multiply_fast function

```
u = [1,2,3]
v = [4,5]
b = 10
result = multiply_fast(u, v, b)
print("result = ", result)

result = [5, 5, 3, 5]
```

Рис. 11: result

Реализация Алгоритма деления неотрицательных целых чисел

```
def rev_to_int(arr: list[int]) -> int:
    value = 0
    for i in range(len(arr) - 1, -1, -1):
        value = value * b + arr[i]
    return value

def int_to_rev(x: int) -> list[int]:
    if x == 0:
        return [0]
    digits = []
    while x > 0:
        digits.append(x % b)
        x //= b
    return digits

def get_digit(arr: list[int], i: int) -> int:
    return arr[i] if 0 <= i < len(arr) else 0
```

Рис. 12: helper functions

```
def division(u: list[int], v: list[int], b: int) -> tuple[list[int], list[int]]:
```

```
    u = trim_leading_zeros(u)
```

```
    v = trim_leading_zeros(v)
```

```
    if v == [0]:
```

```
        return None
```

```
    u_rev = list(reversed(u)) # [4,3,2,1]
```

```
    v_rev = list(reversed(v))
```

```
    n = len(u_rev) - 1
```

```
    t = len(v_rev) - 1
```

```
    q_rev = [0] * (n - t + 1)
```

```
    u_value = rev_to_int(u_rev) # [1,2,3,4] -> 1234
```

```
    v_value = rev_to_int(v_rev) # [5,6] -> 56
```

```
    # Шаг 2:
```

```
    # пока u >= v * b^(n-t) выполнять:
```

```
    #   q_(n-t) := q_(n-t) + 1
```

```
    #   u := u - v * b^(n-t)
```

```
    while u_value >= v_value * (b ** (n - t)):
```

```
        q_rev[n - t] = q_rev[n - t] + 1
```

```
        u_value = u_value - v_value * (b ** (n - t))
```

```
        u_rev = int_to_rev(u_value)
```

```
    # Шаг 3: для i = n, n-1, ..., t+1 выполнять пункты 3.1 - 3.4
```

```
    for i in range(n, t, -1):
```

```
        u_i = get_digit(u_rev, i)
```

```
        u_i_minus_1 = get_digit(u_rev, i - 1)
```

Рис. 13: division function

```

# Шаг 3: для i = n, n-1, ..., t+1 выполнять пункты 3.1 - 3.4
for i in range(n, t, -1):
    u_i = get_digit(u_rev, i)
    u_i_minus_1 = get_digit(u_rev, i - 1)
    v_t = get_digit(v_rev, t)

    q_index = i - t - 1

    if u_i > v_t:
        q_rev[q_index] = b - 1
    else:
        q_rev[q_index] = (u_i * b + u_i_minus_1) // v_t

# Шаг 3.2
# пока q_(i-t-1) * (v_t*b + v_(t-1)) > u_i*b^2 + u_(i-1)*b + u_(i-2)
# выполнять q_(i-t-1) := q_(i-t-1) - 1
v_t_minus_1 = get_digit(v_rev, t - 1)
u_i_minus_2 = get_digit(u_rev, i - 2)

while q_rev[q_index] * (v_t * b + v_t_minus_1) > (u_i * b * b + u_i_minus_1 * b + u_i_minus_2):
    q_rev[q_index] = q_rev[q_index] - 1

# Шаг 3.3
# u := u - q_(i-t-1) * b^(i-t-1) * v
u_value = u_value - q_rev[q_index] * (b ** (i - t - 1)) * v_value
u_rev = int_to_rev(abs(u_value)) if u_value != 0 else [0]

# Шаг 3.4
# если u < 0, то
# u := u + v * b^(i-t-1)
# q_(i-t-1) := q_(i-t-1) - 1
if u_value < 0:
    u_value = u_value + v_value * (b ** (i - t - 1))
    q_rev[q_index] = q_rev[q_index] - 1
    u_rev = int_to_rev(u_value)

# Шаг 4: r := u
r_rev = int_to_rev(u_value)

q = trim_leading_zeros(list(reversed(q_rev)))
r = trim_leading_zeros(list(reversed(r_rev)))

```

Рис. 14: division function

```
u = [1, 2, 3, 4]
v = [5, 6]
b = 10

q, r = division(u, v, b)
print("q =", q)
print("r =", r)

q = [2, 2]
r = [2]
```

Рис. 15: result

Выводы

Реализовал алгоритмов многократной точности для целочисленной арифметики.

Список литературы

https://en.wikipedia.org/wiki/Arbitrary-precision_arithmetic